

11. Put the paper keyboard on the table. Focus the webcam on the keyboard.

Running the program for the first time

Running the program is not easy unless you calibrate the keyboard as explained below.

To use the virtual keyboard, calibration is to be done as per the image size (390*220 pixels) used in this project. That is, when you focus the webcam on the paper, the paper keyboard should be displayed properly on the program output screen. The program can be changed to work with other sizes also.

When the virtualLexe program is executed, the video image captured by the webcam opens on your monitor screen. This window is used for calibration and also identifies the user's finger. Adjust the camera position such that the keyboard lies in between the calibration red lines of the screen as shown in Fig. 4.

How it works

First of all, a frame is captured from the webcam. This frame is converted into a binary image, which is then processed to identify the finger or pen. When a finger touches the text on the paper keyboard, its position is identified and shown on the screen. The text

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download on www.efymag.com website.

is written on the text editor as shown in Fig. 2 (which shows the text "This is a virtual keyboard").

For proper result, it is recommended that you place the paper keyboard on a desk or table, with a webcam fixed on a support facing down towards the paper keyboard.

The author is a fourth-year B.Tech. student of electronics & communication engineering. He likes electronics hardware and software projects

VISUAL STUDIO CODE

```
#include <highgui.h>
#include <cv.h>
#include <stdio.h>
#include <windows.h>
#include <windowsx.h>

int key_value[6][11];
void keyboard()
{
    int max_buffer=0,i,j=0,k=0,pen_pixels=0,loc=0;
    cvInitLineIterator(im_bw,cvPoint(170,k),iterator,8,0);
    for(j=0; j<max_buffer; j++)
    {
        if(iterator.ptr[j]==0)
        {
            loc=j+1;
            if((loc-loc)==1)
            {
                if(counter==0)
                {
                    init=loc-1;
                    counter++;
                    pen_pixels=pen_pixels+1;
                }
                loc=loc;
                CV_NEXT_LINE_POINT(iterator);
            }
            if(pen_pixels>6&&pen_pixels<25)
            {
                cvCircle(im_rgb, cvPoint(init*2,k),10,CV_RGB(255,0,0),2,8,0);
                tip_x=init*2;
                tip_y=k+18;
            }
            cvCircle(im_rgb, cvPoint(tip_x*2,k),10,CV_RGB(0,255,0),2,8,0);
            tip_x=tip_x/34;
            if(frame==5)
            {
                frame=0;
                if((last_x==tip_x)&&(last_y==tip_y))
                {
                    keybd_event(key_value[tip_y][tip_x-1],0x45,0,0);
                    frame=0;
                }
                if(frame==1)
                {
                    last_x=tip_x;
                    last_y=tip_y;
                }
                cvShowWindow("Video", im_rgb);
                c = cvWaitKey(1);
                if(c==27)
                break;
            }
            cvReleaseCapture(&capture);
        }
    }
}

cvDestroyWindow("Video");
//cvDestroyWindow("Videol");
}

void keyboard()
{
    key_value[1][1]=0x08;
    key_value[1][2]=0x08;
    key_value[1][3]=0x08;
    key_value[1][4]=0x08;
    key_value[1][5]=0x20;
    key_value[1][6]=0x20;
    key_value[1][7]=0x20;
    key_value[1][8]=0x14;
    key_value[1][9]=0x14;
    key_value[1][10]=0x14;

    key_value[2][1]=0x18;
    key_value[2][2]=0x4E;
    key_value[2][3]=0x4E;
    key_value[2][4]=0x4D;
    key_value[2][5]=0x4E;
    key_value[2][6]=0x42;
    key_value[2][7]=0x56;
    key_value[2][8]=0x43;
    key_value[2][9]=0x58;
    key_value[2][10]=0x5A;

    key_value[3][1]=0x00;
    key_value[3][2]=0x4C;
    key_value[3][3]=0x4B;
    key_value[3][4]=0x44;
    key_value[3][5]=0x48;
    key_value[3][6]=0x47;
    key_value[3][7]=0x46;
    key_value[3][8]=0x44;
    key_value[3][9]=0x53;
    key_value[3][10]=0x41;

    key_value[4][1]=0x50;
    key_value[4][2]=0x4F;
    key_value[4][3]=0x49;
    key_value[4][4]=0x55;
    key_value[4][5]=0x59;
    key_value[4][6]=0x54;
    key_value[4][7]=0x52;
    key_value[4][8]=0x55;
    key_value[4][9]=0x57;
    key_value[4][10]=0x51;

    key_value[5][1]=0x30;
    key_value[5][2]=0x39;
    key_value[5][3]=0x38;
    key_value[5][4]=0x37;
    key_value[5][5]=0x36;
    key_value[5][6]=0x35;
    key_value[5][7]=0x34;
    key_value[5][8]=0x33;
    key_value[5][9]=0x32;
    key_value[5][10]=0x31;
}
```